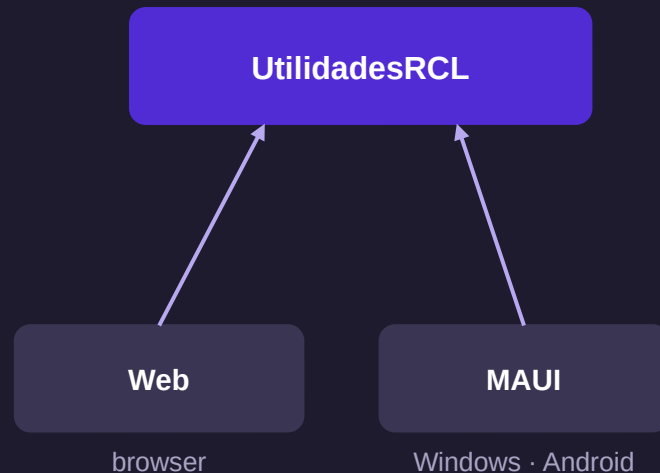


Ficha 2 · parte 3

Razor Class Library

Tirar das duas apps o código repetido e escrever as utilidades uma única vez: dados, serviços e componentes numa biblioteca partilhada.

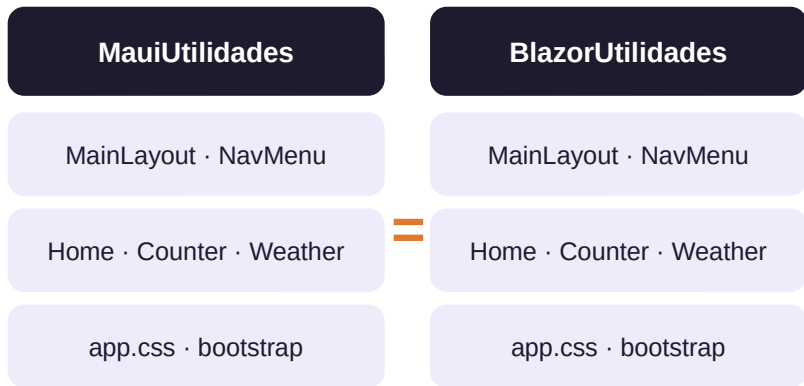


A parte 3, etapa a etapa

Etapa	Alíneas	Passos	Conceitos
1 · Criar a RCL	3a	criar o projeto; referenciá-lo nas duas apps	RCL, as cinco ligações
2 · Despir as apps	3b	mover wwwroot; acertar caminhos; mover NavMenu; apagar exemplos	wwwroot, página anfitriã, _content/
3 · Dados e serviços	3c, 3d	pastas; DTO, interfaces e serviços; registo	DTO, record, async, injeção de dependências
4 · Componentes	3e, 3f	seis componentes; menus; _Imports; router	ciclo de vida, parâmetros
5 · Compilar e correr	3g a 3j	Web, Windows e Android; os dois juntos	
No fim		Program.cs e MauiProgram.cs	arranque e pipeline HTTP

Código completo de cada ficheiro: Ficha2_Parte3_Codigo.md, pela mesma ordem. Cada etapa acaba a compilar.

Duas apps, o mesmo código: a RCL



Cada alteração tem de ser feita duas vezes.

UtilidadesRCL

a solução: um terceiro projeto com o que é comum

Razor Class Library

Uma biblioteca .NET que, além de classes, pode ter componentes Razor e ficheiros estáticos. Compila para uma DLL que as apps referenciam.

- componentes .razor e o seu CSS isolado
- DTO, interfaces e serviços
- ficheiros estáticos em wwwroot

Não é uma aplicação

Não arranca sozinha: não tem Program.cs nem contentor de serviços. Os serviços que lá estão registam-se em cada app.

Criar a RCL e ligá-la às apps

1

Adicionar o projeto

Clique direito na solução > Adicionar > Novo Projeto > Biblioteca de Classes Razor.

2

Nome e versão

UtilidadesRCL, .NET 8. Desmarcar Suportar páginas e vistas: é para MVC, e aqui só há componentes.

3

Limpar

Apagar Component1.razor, Component1.razor.css e ExampleJsInterop.cs.

4

Referenciar

Nas duas apps: Dependências > Adicionar Referência de Projeto > UtilidadesRCL.

Sem o passo 4: erro CS0246, tipo ou namespace não encontrado.

AS CINCO LIGAÇÕES ENTRE AS APPS E A RCL

1

Referência de projeto **etapa 1**

`.csproj`

2

Ficheiros estáticos **etapa 2**

`index.html` · `App.razor`

3

@using **etapas 2 e 4**

`_Imports.razor`

4

Serviços **etapa 3**

`MauiParam.cs` · `Program.cs`

5

Rotas **etapa 4**

`Routes.razor` · `Program.cs`

O que é o wwwroot

É a raiz web do projeto: a pasta dos ficheiros estáticos. Não são compilados; chegam ao browser (ou à WebView) tal como estão.

Na app Web

O `UseStaticFiles()` do `Program.cs` serve a pasta na raiz do site: `wwwroot/app.css` responde em `/app.css`.

Na app MAUI

Os ficheiros vão dentro da app instalada. A `BlazorWebView` serve-os e abre o `wwwroot/index.html`.

Na RCL

Não há servidor. O `wwwroot` da biblioteca aparece nas apps que a usam, em `_content/UtilidadesRCL/`.

O que costuma lá estar

- CSS: o Bootstrap e o `app.css`
- imagens e ícones, como o `favicon.png`
- JavaScript e tipos de letra

O que não está lá

O CSS isolado (`NavMenu.razor.css`, `MainLayout.razor.css`) vive ao lado do componente. O build junta-o num ficheiro gerado, `{Projeto}.styles.css`.

Os ficheiros do wwwroot, um a um

Ficheiro	O que é	Onde está	Destino
<code>bootstrap/bootstrap.min.css</code>	framework CSS: grelha, botões, tabelas (table, btn...)	Web; no MAUI em css/	move para a RCL
<code>bootstrap.min.css.map</code>	mapa para ler o CSS minificado no F12	ao lado do anterior	move com ele
<code>app.css</code>	estilos globais: layout base, erros, validação	Web; no MAUI em css/	move para a RCL
<code>favicon.png</code>	o ícone do separador do browser	Web e MAUI	move para a RCL
<code>index.html</code>	a página que a BlazorWebView abre	só no MAUI	fica no MAUI
<code>background.png</code> · <code>exampleJsInterop.js</code>	exemplos do template da RCL	RCL	fica (figura 29)

Os ficheiros do MAUI e do Web são praticamente iguais: move-se um conjunto e apaga-se o outro.

A página anfitriã: onde o Blazor arranca

MAUI

MainPage.xaml (fica)

```
<BlazorWebView x:Name="blazorWebView"
               HostPage="wwwroot/index.html">
  <BlazorWebView.RootComponents>
    <RootComponent Selector="#app"
                   ComponentType="{x:Type local:Components.Routes}" />
  </BlazorWebView.RootComponents>
</BlazorWebView>
```

wwwroot/index.html (fica)

```
<div id="app">Loading...</div>
<script src="_framework/blazor.webview.js"
        autostart="false"></script>
```

A MainPage abre o index.html e desenha o Routes dentro do `<div id="app">`.

Web

Components/App.razor (fica)

```
<head>
  <link rel="stylesheet" href="..." />
</head>
<body>
  <Routes @rendermode="InteractiveServer" />
  <script src="_framework/blazor.web.js"></script>
</body>
```

O App.razor é o próprio documento HTML. Tem o papel do index.html do MAUI.

Por isso ficam nas apps: cada anfitrião arranca o Blazor à sua maneira. O que é desenhado lá dentro vem da RCL.

MauiUtilidades, ficheiro a ficheiro

Ficheiro	O que é	Destino	Porquê
Platforms/ · Resources/	código e recursos nativos: ícone, splash, fontes	fica	só existem numa app instalada
App.xaml · MainPage.xaml	a app nativa e a página que aloja a BlazorWebView	fica	é o anfitrião
MauiProgram.cs	arranque da app e registo de serviços	fica	cada app tem o seu contentor
wwwroot/index.html	a página HTML que a BlazorWebView abre	fica	o MainPage aponta para ela
wwwroot/css/ · favicon.png	Bootstrap, app.css e ícone	apaga	passam a vir da RCL
Components/Routes.razor	o router das páginas	fica	acrescenta-se a RCL
Components/_Imports.razor	@using dos .razor desta app	fica	ajusta-se
Layout/MainLayout.razor	a moldura: menu à esquerda, página à direita	fica	o layout é da app
Layout/NavMenu.razor	o menu lateral	move	para UtilidadesRCL/Shared (criar a pasta)
Pages/Home.razor	a página inicial "/"	fica	passa a usar o HomeComponent
Pages/Counter · Weather	exemplos do template	apaga	dão lugar às utilidades

BlazorUtilidades, ficheiro a ficheiro

Ficheiro	O que é	Destino	Porquê
launchSettings.json	perfis de arranque: portas http e https	fica	é da app Web
appsettings.json (e .Development)	configuração: logging, hosts permitidos	apaga	a figura 28 não o mostra
Program.cs	arranque, serviços e pipeline HTTP	fica	explicado no fim da aula
wwwroot/ (bootstrap, app.css, favicon)	CSS e ícone	move	passam para a RCL
Components/App.razor	o documento HTML: head, CSS, script	fica	é a página anfitriã da Web
Components/Routes.razor	o router das páginas	fica	acrescenta-se a RCL
Components/_Imports.razor	@using dos .razor desta app	fica	ajusta-se
Layout/MainLayout.razor	a moldura da página	fica	o layout é da app
Layout/NavMenu.razor	o menu lateral	apaga	usa o da RCL (a ficha deixava-o)
Pages/Home.razor	a página inicial "/"	fica	passa a usar o HomeComponent
Pages/Counter · Weather · Error	exemplos e página de erro	apaga	a figura 28 só deixa o Home

Como devem ficar as duas apps

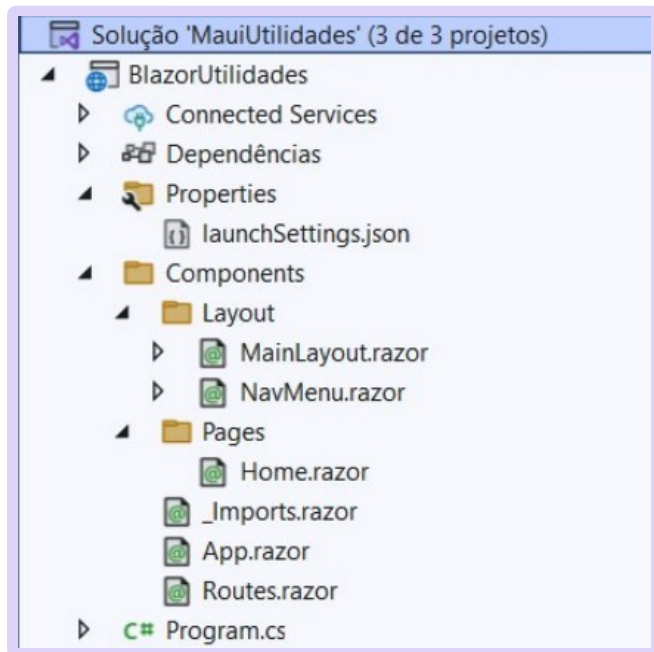
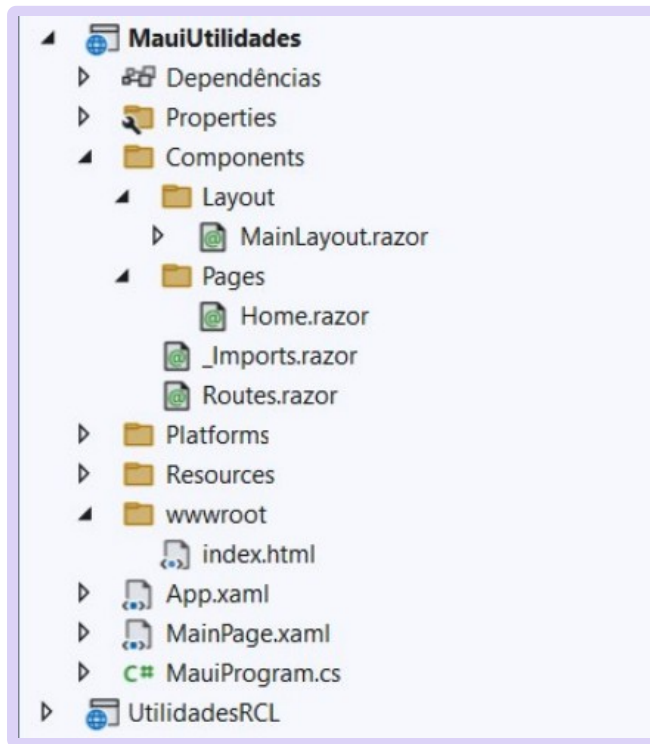


Figura 28 da ficha



Web

Saíram o wwwroot, o appsettings.json, o Counter, o Weather e o Error. O NavMenu também sai: usa o da RCL (na figura ainda aparece).

MAUI

Saíram o css/ e o favicon, o Counter e o Weather. O NavMenu foi para a RCL. Fica o index.html.

A alínea 3b, por esta ordem

1

Mover o wwwroot

O do Web vai para a RCL; o do MAUI apaga-se, exceto o index.html.

2

Acertar os caminhos

index.html (MAUI) e App.razor (Web) passam a apontar para _content/UtilidadesRCL/.

3

Um só NavMenu, na RCL

Pasta Shared na RCL; o do MAUI vai para lá, o do Web apaga-se; @using nas duas apps.

4

Apagar o que a figura 28 não mostra

Counter e Weather (e as entradas do menu); no Web, também Error e appsettings.

5

Compilar e correr

Ponto de controlo: as duas apps com estilos, menu e página Home.

O que não muda

MainLayout, Routes e Home ficam nos dois projetos; os dois últimos só mudam na etapa 4. O NavMenu passa a ser um só, na RCL.

Porquê por esta ordem

Cada passo deixa a solução a compilar. Os exemplos apagam-se no fim, para haver sempre um ponto de retorno.

Mover o wwwroot

MauiUtilidades/wwwroot

- **index.html** fica
- css/ apagar
- favicon.png apagar

BlazorUtilidades/wwwroot

- bootstrap/
- app.css
- favicon.png

A pasta fica vazia e pode apagar-se.

mover

UtilidadesRCL/wwwroot

- bootstrap/
- app.css
- favicon.png
- e o que já vinha do template

Porque se move o do Web

Já está na raiz do wwwroot, como na figura 29. Os do MAUI são cópias quase iguais dentro de css/, por isso apagam-se.

Atenção à pasta css/

No MAUI os caminhos tinham css/. Na RCL os ficheiros ficam na raiz do wwwroot, e é esse o caminho que conta no passo seguinte.

Acertar os caminhos: `_content/`

O `wwwroot` de uma biblioteca não se junta à raiz da app. No build, os ficheiros ficam disponíveis num caminho próprio:

```
_content/UtilidadesRCL/app.css
```

- Funciona igual na Web e no MAUI.
- O nome no caminho é o do assembly da RCL.
- O ficheiro não é copiado: é servido a partir da RCL.

Se falhar

A página aparece sem Bootstrap. No browser, F12 > Rede mostra 404 nos `.css`.

Mudam três linhas, iguais nos dois ficheiros

```
MAUI: wwwroot/index.html · Web: Components/App.razor

<link rel="stylesheet"
      href="_content/UtilidadesRCL/bootstrap/bootstrap.min.css" />
<link rel="stylesheet"
      href="_content/UtilidadesRCL/app.css" />
<link rel="icon" type="image/png"
      href="_content/UtilidadesRCL/favicon.png" />
```

Antes: `css/bootstrap/bootstrap.min.css` e `css/app.css` no MAUI;
`bootstrap/bootstrap.min.css` e `app.css` na Web.

A linha do `{Projeto}.styles.css` não muda: é gerada no build, junta o CSS isolado e já importa o da RCL.

Um só NavMenu, em Shared

Criar na RCL a pasta Shared e arrastar para lá, com Shift para mover, NavMenu.razor e NavMenu.razor.css, de MauiUtilidades/Components/Layout.

O namespace de um componente é a pasta onde está

UtilidadesRCL/Shared/NavMenu.razor é a classe UtilidadesRCL.Shared.NavMenu. O MainLayout de cada app só a encontra com um @using.

```
_Imports.razor do MAUI e do Web, no fim  
@using UtilidadesRCL.Shared
```

E o NavMenu do Web?

Apaga-se: era uma cópia igual (a ficha deixava-o). No da RCL, o cabeçalho passa de MauiUtilidades a Utilidades.

UtilidadesRCL/_Imports.razor, por agora

```
@using Microsoft.AspNetCore.Components.Web  
@using Microsoft.AspNetCore.Components.Routing
```

Porquê o Routing na RCL

O NavLink que o menu usa vive nesse namespace. O template da RCL só traz o Components.Web.

Sintoma se faltar um @using

Aviso RZ10012: o NavMenu ou o NavLink aparecem como tags HTML desconhecidas. O menu fica vazio ou os links não navegam.

Apagar o resto e compilar

MauiUtilidades/Components/Pages

- Counter.razor · Weather.razor apagar
- Home.razor fica

BlazorUtilidades/Components/Pages

- Counter.razor · Weather.razor apagar
- Error.razor apagar
- Home.razor fica

E no NavMenu da RCL

Tirar as entradas Counter e Weather. Senão, os links apontam para páginas que já não existem e dão erro.

PONTO DE CONTROLO

- 1 UtilidadesRCL criada e referenciada pelas duas apps
- 2 wwwroot movido; no MAUI fica só o index.html
- 3 _content/UtilidadesRCL/ no index.html e no App.razor
- 4 NavMenu só na RCL; @using ...Shared nas duas apps
- 5 @using ...Components.Routing no _Imports da RCL
- 6 Counter e Weather apagados, também do menu

Teste: compilar (Ctrl+Shift+B) e correr as duas apps. Abrem com estilos e o menu só com o Home.

A etapa 3, por esta ordem

1

Criar as pastas

Data/DTO, Data/Interfaces, Data/Services e Pages

3c

2

Temperatura

DTO, interface e serviço, em conjunto

3d

antes: DTO, record, interface, async

3

Energia, Evento e Notícia

o mesmo molde, cada um por si

3d

4

Registar os serviços

quatro linhas no MauiProgram.cs e no Program.cs

fora da ficha

antes: injeção de dependências

5

Compilar

ponto de controlo, antes dos componentes

Os conceitos entram imediatamente antes do passo que precisa deles.

Estrutura da UtilidadesRCL

```
UtilidadesRCL/  
├── wwwroot/  
├── Data/  
│   ├── DTO/  
│   │   ├── Energia, Evento,  
│   │   │   Noticia, Temperatura  
│   │   └── Interfaces/  
│   │       ├── IEnergiaService, ...  
│   │       └── Services/  
│   │           ├── EnergiaService, ...  
│   │           └── Pages/  
│   │               ├── EnergiaComponent, ...  
│   │               ├── HomeComponent, MenuComponent  
│   └── Shared/  
│       └── NavMenu.razor  
└── _Imports.razor
```

Cada pasta é um namespace: UtilidadesRCL.Data.DTO, UtilidadesRCL.Data.Interfaces, ... Por isso criam-se as pastas antes dos ficheiros.

DTO

O formato dos dados.

Interfaces

O que cada serviço promete.

Services

Como o cumpre, de facto.

Pages

O que o utilizador vê.

Shared

Peças comuns, como o menu.

Uma pasta por responsabilidade.

namespace: o nome completo de uma classe

Data/DTO/Temperatura.cs

```
namespace UtilidadesRCL.Data.DTO;  
  
public record Temperatura(  
    string Dia, string TemperaturaC, string Resumo);
```

O nome verdadeiro da classe:

Temperatura



UtilidadesRCL.Data.DTO.Temperatura

Duas formas de escrever

```
// vale para o ficheiro todo (DTO, serviços)  
namespace UtilidadesRCL.Data.DTO;  
  
// forma antiga, com chavetas (MauiProgram.cs)  
namespace MauiUtilidades  
{  
    public static class MauiProgram { ... }  
}
```

Para que serve

- **Organizar:** DTO, interfaces e serviços, cada um no seu namespace.
- **Evitar choques de nomes:** duas classes Temperatura convivem, se estiverem em namespaces diferentes.

Convenção: segue as pastas

O Visual Studio põe o namespace do projeto e da pasta ao criar o ficheiro. Para o compilador conta a linha namespace, não a pasta.

E nos .razor?

Não há linha namespace: vem da pasta. Por isso o NavMenu, em Shared, passou a ser UtilidadesRCL.Shared.NavMenu.

using: um atalho para o nome completo

Sem using: o nome completo

```
namespace UtilidadesRCL.Data.Interfaces;
public interface ITemperaturaService
{
    Task<IEnumerable<UtilidadesRCL.Data.DTO.Temperatura>>
        LoadTemperaturasAsync();
}
```

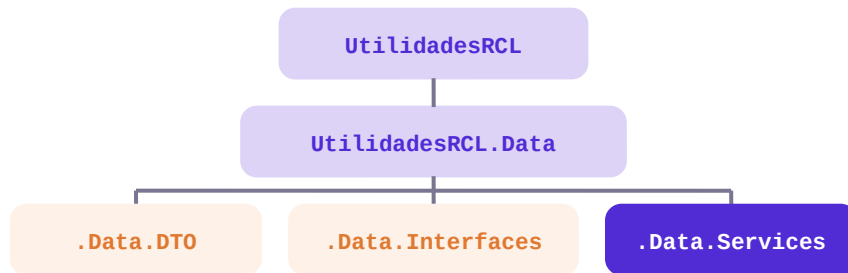
Com using: como está no ficheiro

```
using UtilidadesRCL.Data.DTO;

namespace UtilidadesRCL.Data.Interfaces;
public interface ITemperaturaService
{
    Task<IEnumerable<Temperatura>>
        LoadTemperaturasAsync();
}
```

O using não traz código: quem torna a RCL disponível é a referência de projeto. Por isso o CS0246 tem duas causas: falta a referência ou falta o using.

O que o TemperaturaService vê sem using



Vê o seu namespace e os de cima. Os do lado (DTO e Interfaces) precisam de using.

Já incluídos: o .csproj tem ImplicitUsings ligado, que traz System, System.Linq, System.Collections.Generic e System.Threading.Tasks. Por isso Task e IEnumerable funcionam sem using.

Nos .razor: @using é o mesmo. O _Imports.razor junta os @using dos .razor da pasta, mas não chega aos .cs.

DTO: objetos só com dados

Data Transfer Object. Um objeto sem lógica, só com as propriedades que um lado precisa de enviar ao outro.

Entidades: como estão na base de dados (ex.: Gestão de Loja)

```
public class Produto
{
    public int Id { get; set; }
    public string Nome { get; set; }
    public decimal Preco { get; set; }
    public decimal PrecoCusto { get; set; }
    public int CategoriaId { get; set; }
    public Categoria Categoria { get; set; }
}

public class Categoria
{
    public int Id { get; set; }
    public List<Produto> Produtos { get; set; }
}
```

Na ficha 2 ainda não há base de dados. A Temperatura já é um DTO porque é esse o papel que vai ter quando os dados vierem da API.

DTO: o que o ecrã precisa

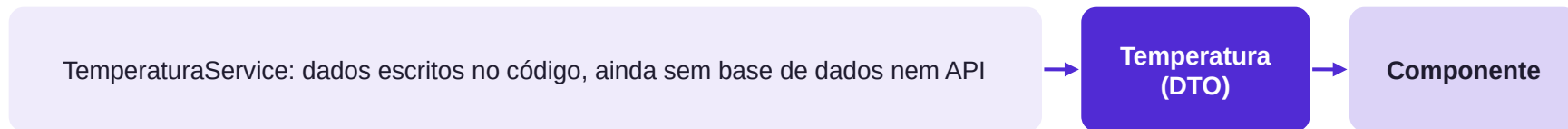
```
public record ProdutoDto(
    int Id,
    string Nome,
    decimal Preco,
    string Categoria);
```

Porquê separar

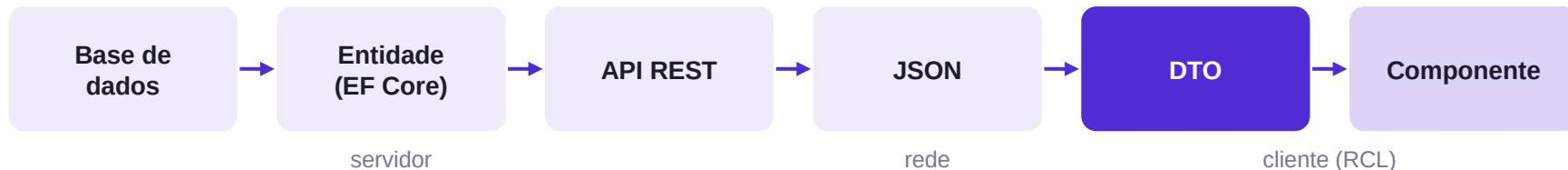
- Esconder o que é interno: o PrecoCusto nunca chega ao browser.
- Evitar ciclos: Produto > Categoria > Produtos > ... rebenta a conversão para JSON.
- Dar ao ecrã a forma que lhe dá jeito: o nome da categoria, não o objeto.
- Mudar a base de dados sem mudar os componentes.

Onde entram os DTO no caminho dos dados

HOJE (FICHA 2)



MAIS À FRENTE (FICHAS 3 A 8)



O componente só conhece o DTO. Por isso, quando o serviço passar a ir buscar os dados à API, a página fica igual: só muda o que está à esquerda.

record: uma classe de dados numa linha

Data/DTO/Temperatura.cs

```
public record Temperatura(  
    string Dia,  
    string TemperaturaC,  
    string Resumo);
```

init: as propriedades só se atribuem ao criar o objeto.

Igualdade por valor: dois records com os mesmos dados são iguais.

with: cópia com uma alteração
`var t2 = t1 with { Resumo = "Frio" };`

Aproximadamente o que o compilador gera

```
public class Temperatura  
{  
    public string Dia { get; init; }  
    public string TemperaturaC { get; init; }  
    public string Resumo { get; init; }  
  
    public Temperatura(string Dia,  
        string TemperaturaC, string Resumo)  
    {  
        this.Dia = Dia;  
        this.TemperaturaC = TemperaturaC;  
        this.Resumo = Resumo;  
    }  
    // + Equals e GetHashCode por valor,  
    // ToString, Deconstruct e with  
}
```

Na solução todos os campos são string porque só são mostrados. Numa app real, Dia seria DateTimeOffset e a temperatura um número.

class, record e struct

	class	record	struct
Tipo	referência	referência	valor (copiado ao atribuir)
a == b quando	é o mesmo objeto	têm os mesmos valores	não compila, a não ser que se defina
Alterar depois de criar	sim, com set	não, na forma posicional (init)	sim, com set
Herança	sim	sim, entre records	não
Uso típico	entidades, serviços	DTO, mensagens, resultados	valores pequenos: ponto, cor

O mesmo record, escrito com propriedades

```
public record Temperatura
{
    public required string Dia { get; init; }
    public required string TemperaturaC { get; init; }
    public string Resumo { get; init; } = "";
}
```

Porquê record para um DTO

É só dados, vem de fora e não deve mudar depois de criado. Comparar por valor dá jeito em testes.

O C# que aparece nesta ficha

Sintaxe	O que faz	Onde aparece
<code>namespace X;</code>	vale para o ficheiro todo; poupa um nível de chavetas	DTO, interfaces, serviços
<code>string? IEnumerable<T>?</code>	pode ser null; sem ?, o compilador avisa se puder ser	componentes
<code>new[] { ... }</code>	array com o tipo deduzido dos elementos	serviços
<code>async · await · Task<T></code>	método que espera sem bloquear; Task é a promessa	serviços, componentes
<code>IEnumerable<T></code>	qualquer coisa que se percorre com foreach	interfaces
<code>partial class</code>	classe repartida por dois ficheiros	MainPage.xaml.cs, code-behind
<code>[Parameter]</code>	atributo: valor que vem de quem usa o componente	HomeComponent
<code>x ?? y</code>	se x for null, usa y	ficha 6 (API)
<code>=> expressão</code>	corpo de uma linha, em métodos e propriedades	construtores curtos

Interface: o que o serviço promete

```
Data/Interfaces/ITemperaturaService.cs (completo)
using UtilidadesRCL.Data.DTO;

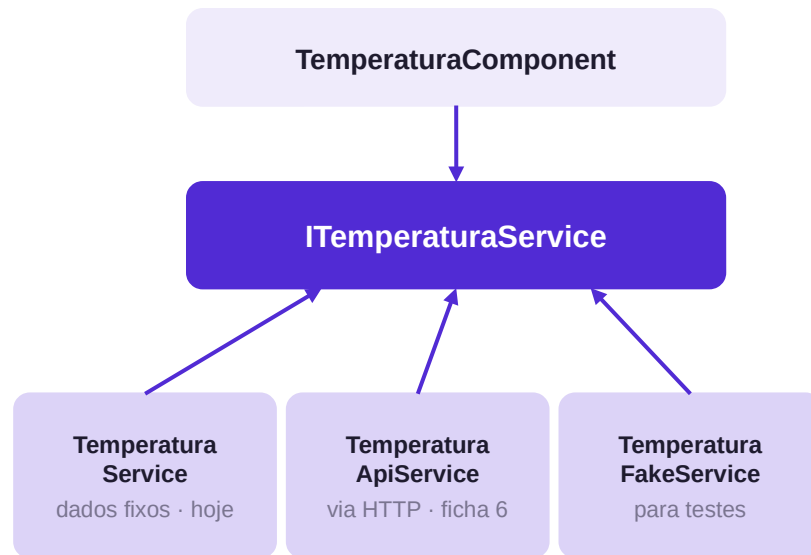
namespace UtilidadesRCL.Data.Interfaces;

public interface ITemperaturaService
{
    Task<IEnumerable<Temperatura>>
        LoadTemperaturasAsync();
}
```

`Task<...>` o resultado chega mais tarde

`IEnumerable<T>` algo que se percorre, não uma List

`...Async();` sem corpo: diz o quê, não o como



O componente só conhece a interface. Qual das três recebe decide-se no registo (etapa 3).

A solução do regente escreve `Task<...>?` com ponto de interrogação: dá o aviso CS8602 no await. Aqui fica sem ele.

Porque é que há esperas

Numa app, quase todo o tempo é passado à espera de algo que está fora do processador: a rede, o disco, a base de dados.

Operação	Tempo real		Se 1 ns fosse 1 s
Uma instrução do processador	~1 ns		1 segundo
Ler da memória RAM	~100 ns		~2 minutos
Ler de um disco SSD	~100 µs		~1 dia
Pedido a uma API na Internet	~100 ms		~3 anos
O nosso Task.Delay(1000)	1 s		~32 anos

Barras em escala logarítmica: cada passo para a direita é 10 vezes mais tempo. Valores aproximados.

Threads: quem executa o código

O que é uma thread

Uma linha de execução do programa. O processador corre as várias threads à vez, e quem decide a vez de cada uma é o sistema operativo.

Uma app tem sempre pelo menos uma; um servidor tem muitas.

Uma thread bloqueada

Existe, ocupa memória e não faz nada: está parada à espera, e não pode ser usada para mais nada.

É o que acontece quando um método síncrono espera pela rede, ou quando se usa `.Result` numa Task por acabar.

App MAUI: uma só thread de interface

clique

desenha

bloqueada 3 s

...

Se bloquear, a janela congela: não redesenha nem responde. Ao fim de uns 5 s, o Windows diz “Não está a responder” e o Android mostra “A app não está a responder”.

App Web: um conjunto de threads para todos

thread 1: a trabalhar para A

pedidos em fila:

thread 2: parada à espera (B)

E

F

G

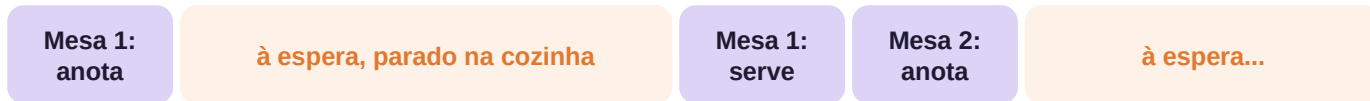
thread 3: parada à espera (C)

thread 4: a trabalhar para D

Cada thread bloqueada é uma a menos para os outros utilizadores. Com muitos, os pedidos ficam em fila.

Síncrono ou assíncrono: o empregado de mesa

Síncrono: espera na cozinha



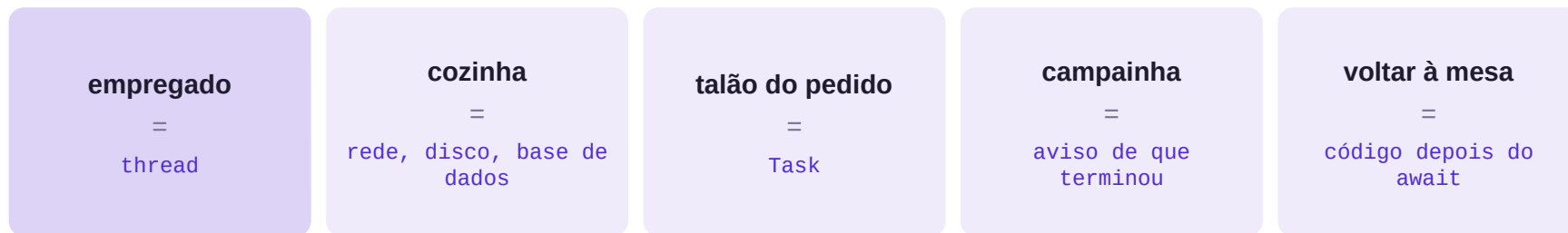
A mesa 2 só é atendida quando o prato 1 fica pronto. A mesa 3 nem chega a ser vista.

Assíncrono: vai a outra mesa



Volta a cada mesa quando a campanha da cozinha toca. Três mesas atendidas no mesmo tempo.

NA APLICAÇÃO



Síncrono ou assíncrono: o mesmo código

Síncrono

```
// a thread fica parada até os dados chegarem  
var dados = servico.LoadTemperaturas();  
Mostrar(dados);
```

Thread

pede

parada à espera

mostra

Rede

o pedido vai e a resposta volta

Assíncrono

```
// a thread fica livre enquanto os dados não chegam  
var dados = await servico.LoadTemperaturasAsync();  
Mostrar(dados);
```

Thread

pede

livre: desenha, responde a cliques

mostra

Rede

o pedido vai e a resposta volta

O tempo total e o resultado são iguais. Muda o que a thread faz durante a espera: no síncrono fica presa sem fazer nada; no assíncrono fica livre para o resto da app, e volta quando a resposta chega.

Task: a promessa de um resultado

Uma Task é um objeto que representa uma operação que ainda pode estar a decorrer. **É o talão do pedido: ainda não é o prato, mas vai dar o prato.**

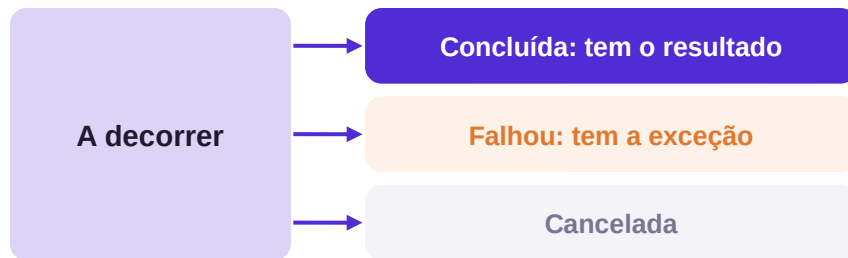
Task operação sem resultado: o void dos métodos async

Task<T> operação que, quando acabar, dá um T

async marca o método: pode usar await e devolve uma Task

await espera pela Task sem bloquear e tira-lhe o resultado

O CICLO DE UMA TASK



Passo a passo (normalmente numa só linha)

```
Task<IEnumerable<Temperatura>> tarefa =  
    servico.LoadTemperaturasAsync(); // o talão  
var temperaturas = await tarefa;    // o prato
```

Se a Task falhar, o await lança a exceção nessa linha, e apanha-se com try/catch como no código síncrono. Por isso o await é a forma certa de ler o resultado: .Result também o dá, mas bloqueia a thread até lá.

O que o await faz, passo a passo

```
async Task<string> LerAsync(string url)
{
    Log("vou pedir");                // parte 1
    var html = await http.GetStringAsync(url);
    Log("chegou");                   // parte 2
    return html;
}
```

- 1 A parte 1 corre na thread, como qualquer código.
- 2 O GetStringAsync envia o pedido e devolve logo uma Task por acabar.
- 3 O await vê que não acabou: o método sai, entrega a sua Task a quem o chamou e a thread fica livre.
- 4 A resposta chega: a placa de rede avisa o sistema operativo e a parte 2 entra na fila.
- 5 Uma thread livre corre a parte 2, com as variáveis como estavam.

Thread

1 parte 1

3 livre: desenhar, outros cliques

5 parte 2

Rede

2 pedido a caminho

4 resposta

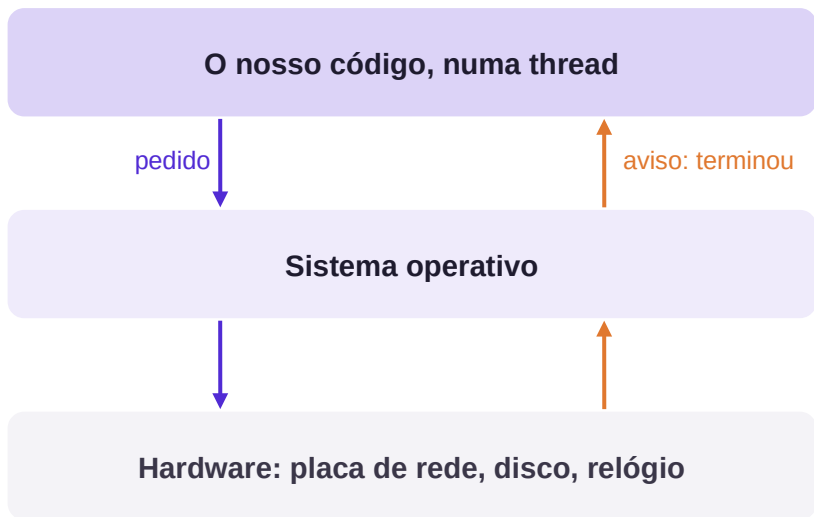
tempo

Quem faz isto é o compilador

Parte o método em dois: o que vem antes do await e o que vem depois (a continuação). Guarda as variáveis num objeto, para a parte 2 continuar onde a parte 1 parou.

Nós só escrevemos async e await, e o código lê-se de cima para baixo como se fosse síncrono.

Por baixo: threads novas ou gestão do CPU?



Enquanto o hardware trabalha, nenhuma thread está à espera deste pedido.

async não é paralelismo: é não ficar parado à espera.

O await lança uma thread nova?

Não. Esperar não precisa de thread: quem trabalha é o hardware, e o sistema operativo avisa quando acaba.

Então quem gere o processador?

O sistema operativo, como sempre: reparte o CPU pelas threads com trabalho. O async só evita threads paradas a ocupar lugar.

Onde corre a parte depois do await?

No MAUI, na thread da interface. Na Web, numa thread livre do conjunto do servidor (thread pool).

E se for trabalho de CPU, contas pesadas?

Aí não há espera, há contas. Task.Run passa-as a outra thread do pool; no MAUI, o ecrã não congela.

Quando usar assíncrono

Regra simples: se o método espera por algo fora do processador (rede, base de dados, disco), é assíncrono.

Situação	Como	Exemplo
Pedir dados a uma API	assíncrono	<code>await http.GetFromJsonAsync<T>(url)</code>
Ler ou gravar na base de dados	assíncrono	<code>await contexto.Produtos.ToListAsync()</code>
Ler ou escrever ficheiros	assíncrono	<code>await File.ReadAllTextAsync(caminho)</code>
Contas rápidas, LINQ em memória	síncrono	<code>lista.Where(p => p.Preco > 10)</code>
Contas pesadas no MAUI	Task.Run	<code>await Task.Run(() => Calcular())</code>

async até ao fim

quem chama um método
async também faz await

Nunca .Result nem .Wait()

bloqueiam a thread e podem
prender a app

Devolver Task, não void

com async void ninguém sabe
quando acaba

O nome acaba em Async

avisa quem chama que deve
fazer await

No nosso código: serviço e componente

```
Data/Services/TemperaturaService.cs
public async Task<IEnumerable<Temperatura>>
    LoadTemperaturasAsync()
{
    await Task.Delay(1000);
    // ... prepara as temperaturas
    return temperaturas;
}
```

O `Task.Delay` faz de conta que os dados vêm de uma API lenta. Quando vierem mesmo da API (ficha 6), troca-se por um `await` ao `HttpClient` e o resto fica igual.

Por isso o componente testa `is null`: da primeira vez que desenha, a lista ainda não chegou.

1

0 s O componente chama `LoadTemperaturasAsync()`.

2

O serviço chega ao `await` e devolve uma `Task` ainda por terminar.

3

O componente desenha já, com “A carregar temperaturas...”.

4

1 s A `Task` termina e o componente volta a desenhá-la, agora com a tabela.

Temperatura: DTO, interface e serviço

Data/DTO/Temperatura.cs

```
namespace UtilidadesRCL.Data.DTO;  
  
public record Temperatura(  
    string Dia, string TemperaturaC, string Resumo);
```

Data/Interfaces/ITemperaturaService.cs

```
using UtilidadesRCL.Data.DTO;  
  
namespace UtilidadesRCL.Data.Interfaces;  
  
public interface ITemperaturaService  
{  
    Task<IEnumerable<Temperatura>>  
        LoadTemperaturasAsync();  
}
```

Data/Services/TemperaturaService.cs

```
using UtilidadesRCL.Data.DTO;  
using UtilidadesRCL.Data.Interfaces;  
  
namespace UtilidadesRCL.Data.Services;  
  
public class TemperaturaService : ITemperaturaService  
{  
    public async Task<IEnumerable<Temperatura>>  
        LoadTemperaturasAsync()  
    {  
        await Task.Delay(1000);  
  
        var temperaturas = new[]  
        {  
            new Temperatura("Segunda", "24", "Ameno"),  
            new Temperatura("Terça", "27", "Quente"),  
            new Temperatura("Quarta", "31", "Muito quente"),  
            new Temperatura("Quinta", "22", "Ameno"),  
            new Temperatura("Sexta", "18", "Fresco"),  
        };  
        return temperaturas;  
    }  
}
```

As outras três: o mesmo molde

Utilidade	DTO (campos)	Interface · método	Serviço	Rota
Energia	Dia, Período, PreçoKWh	IEnergiaService · LoadEnergiasAsync	EnergiaService	/energias
Evento	Data, Nome, Local	IEventoService · LoadEventosAsync	EventoService	/eventos
Notícia	Data, Título, Fonte	INoticiaService · LoadNoticiasAsync	NoticiaService	/noticias
Temperatura	Dia, TemperaturaC, Resumo	ITemperaturaService · LoadTemperaturasAsync	TemperaturaService	/temperaturas

Sugestão para a aula

Fazer a Temperatura em conjunto e deixar as outras três para cada um. Mudam os nomes e os campos; a forma é igual.

Código completo

No Markdown, secção 3, pronto a copiar. Todos os campos são string, como na solução.

O problema: o componente cria o que precisa

Sem injeção de dependências

```
@code {  
    private IEnumerable<Temperatura>? temperaturas;  
  
    protected override async Task  
        OnInitializedAsync()  
    {  
        var servico = new TemperaturaService();  
        temperaturas = await  
            servico.LoadTemperaturasAsync();  
    }  
}
```

Funciona. Mas o componente passou a depender de uma classe concreta, e isso tem custos.

1

Mudar custa caro

Quando os dados vierem da API, é preciso alterar todos os componentes que fazem new TemperaturaService().

2

Dependências em cadeia

Se o serviço precisar de um HttpClient ou da base de dados, o componente também tem de saber criá-los.

3

Sem controlo do tempo de vida

Cada componente cria o seu objeto: não há forma de o partilhar nem de decidir quanto tempo vive.

Acoplamento: quanto uma classe depende de outra

Acoplamento forte

```
private TemperaturaService servico = new();
```

- conhece a classe concreta;
- sabe como a construir, e o que ela precisa;
- mudar uma obriga a mudar a outra;
- não se testa sem a verdadeira.

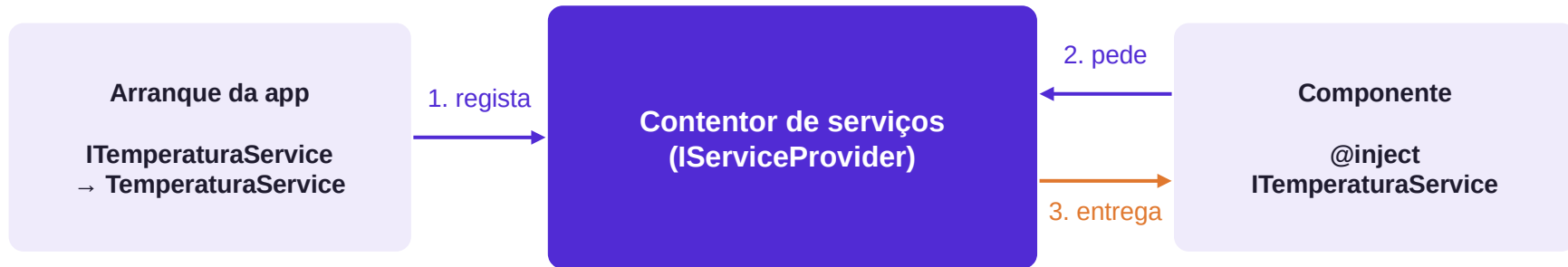
Acoplamento fraco

```
@inject ITemperaturaService servico
```

- conhece só o contrato (a interface);
- não sabe quem a cria, nem como;
- troca-se a implementação sem lhe tocar;
- testa-se com um serviço falso.

O objetivo: partes pouco acopladas entre si, cada uma com uma só responsabilidade. Interfaces e injeção de dependências são a ferramenta.

A ideia: pedir em vez de criar



O componente diz o que precisa: uma interface, ou seja, um contrato. Não diz qual a classe nem como se constrói.

Quem decide a classe, cria o objeto e o entrega é o contentor, configurado uma única vez no arranque. O controlo de criar objetos passa do componente para a aplicação: é a inversão de controlo.

O contentor é uma lista de receitas

No arranque, a aplicação preenche uma lista: para cada contrato, qual a classe a usar e quanto tempo o objeto dura. Essa lista é o contentor.

Quando pedirem...	...o contentor cria	Duração
<code>ITemperaturaService</code>	<code>TemperaturaService</code>	Scoped
<code>IEnergiaService</code>	<code>EnergiaService</code>	Scoped
<code>NavigationManager</code>	(já vem do Blazor)	Scoped
<code>ILogger<T></code>	(já vem do .NET)	Singleton

Muitos serviços já lá estão sem ninguém os registar: vêm do próprio Blazor e do .NET. Os nossos juntam-se a essa lista.

Registado uma vez, usado em todo o lado

Qualquer componente ou serviço pede o que está na lista, sem saber quem o criou.

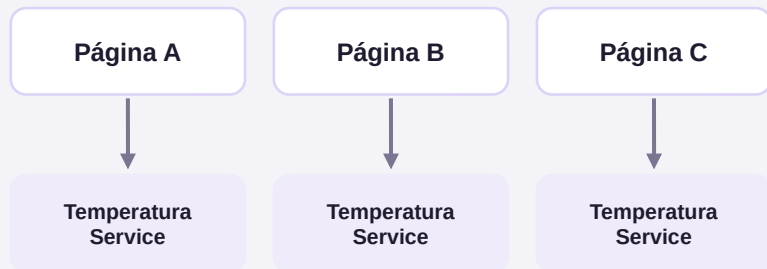
Quando um componente pede

1. O contentor procura o contrato na lista.
2. Vê se já existe um objeto válido para este âmbito.
3. Se não existir, cria-o, resolvendo primeiro o que o construtor dele pedir.
4. Entrega-o ao componente.

Se o contrato não estiver na lista, a aplicação falha ao abrir a página.

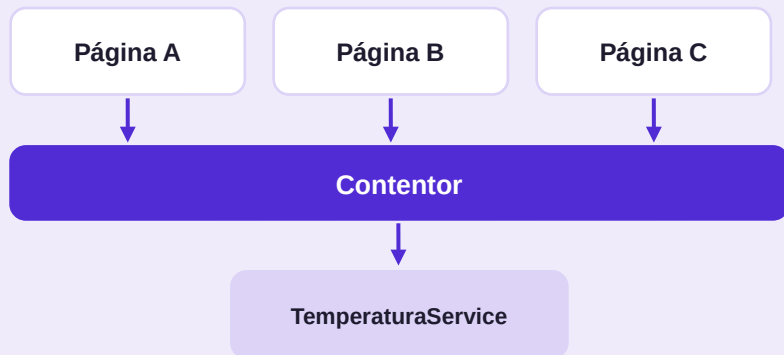
Com e sem injeção de dependências

Sem injeção: cada um constrói o seu



Três objetos diferentes. Para mudar de implementação, é preciso alterar as três páginas.

Com injeção: pedem ao contentor



Um objeto, criado e entregue pelo contentor. Mudar de implementação altera uma linha no arranque.

Trocar a implementação

Dados em memória hoje, API amanhã, sem tocar nas páginas.

Partilhar o mesmo objeto

Páginas diferentes podem trabalhar sobre os mesmos dados.

Decidir num sítio só

O arranque da app define o que se usa e quanto tempo dura.

Em .NET são três passos

1 Registrar

No arranque de cada app, antes do `builder.Build()`.

```
builder.Services
    .AddScoped<
        ITemperaturaService,
        TemperaturaService>();
```

2 Pedir

Num componente, com `@inject`. No code-behind, com `[Inject]`.

```
@* no .razor *@
@inject ITemperaturaService
        servico

// no .razor.cs
[Inject]
public ITemperaturaService
    Servico { get; set; }
    = default!;
```

3 Resolver

Numa classe, pede-se pelo construtor. O contentor cria primeiro o que ele pede.

```
public class RelatorioService
{
    private readonly
        ITemperaturaService _t;

    public RelatorioService(
        ITemperaturaService t)
        => _t = t;
}
```

Se faltar um registo em qualquer ponto da cadeia, o erro aparece quando o serviço é pedido, não ao compilar.

Tempos de vida: quanto dura cada objeto

Registo	Uma instância por	Na app Web (Server)	Na app MAUI	Serve para
<code>AddSingleton</code>	aplicação	a mesma para todos os utilizadores	a mesma durante toda a app	configuração, caches
<code>AddScoped</code>	âmbito	uma por circuito, ou seja, por separador	uma por BlazorWebView: na prática, a app	estado do utilizador, acesso a dados
<code>AddTransient</code>	injeção	nova em cada injeção	nova em cada injeção	serviços leves e sem estado

Cuidado com Singleton no servidor

Há um só servidor para todos. Um Singleton que guarde dados de um utilizador mostra-os aos outros.

E numa API (ficha 6)?

Aí o âmbito é o pedido HTTP: um Scoped nasce e morre com cada pedido.

Ver os tempos de vida a funcionar

Data/Services/CicloVidaService.cs (só para a demo)

```
public class CicloVidaService
{
    public string Id { get; } =
        Guid.NewGuid().ToString()[..8];
}
```

Pages/CicloVidaComponent.razor

```
@page "/ciclovida"
@using UtilidadesRCL.Data.Services
@inject CicloVidaService primeiro
@inject CicloVidaService segundo

<h3>Tempos de vida</h3>
<p>Primeira injeção: <strong>@primeiro.Id</strong></p>
<p>Segunda injeção: <strong>@segundo.Id</strong></p>
```

Program.cs: trocar por AddScoped e AddTransient

```
builder.Services.AddSingleton<CicloVidaService>();
```

Registo	As 2 injeções	Outro pedido / separador
AddSingleton	iguais	igual
AddScoped	iguais	diferente
AddTransient	diferentes	diferentes

Testado nesta aplicação

Carregar /ciclovida duas vezes e comparar os códigos. Um serviço pode ser registado sem interface: `AddSingleton<Classe>()`.

Na Web, com Scoped, o código pode mudar logo ao abrir: a pré-renderização e o circuito são âmbitos diferentes.

Singleton: o padrão e o tempo de vida

Padrão de desenho Singleton

```
public sealed class Configuracao
{
    private static readonly Configuracao instancia = new();
    public static Configuracao Instancia => instancia;

    private Configuracao() { }
}

// uso, em qualquer sítio:
var c = Configuracao.Instancia;
```

A própria classe garante que há uma só. Qualquer código lhe chega: estado global escondido, difícil de testar e de trocar.

Singleton pelo contentor

```
public class Configuracao : IConfiguracao
{
    // classe normal, construtor público
}

builder.Services
    .AddSingleton<IConfiguracao, Configuracao>();

// uso:
@Inject IConfiguracao configuracao
```

O registo é que decide que há uma só. Quem a usa pede-a pela interface; num teste, regista-se outra.

Nos dois casos, numa app Web: partilhado por todos os utilizadores e por pedidos em simultâneo.

Quem pode depender de quem

Um serviço...	pede Singleton	pede Scoped	pede Transient
Singleton	sim	não: dá erro	cuidado: fica preso
Scoped	sim	sim	sim
Transient	sim	sim	sim

A regra

Um serviço só deve depender de serviços que vivam tanto ou mais do que ele. Se não, o de vida curta fica preso dentro do de vida longa: é uma dependência cativa.

Exemplo errado

```
builder.Services.AddScoped<ITemperaturaService,  
    TemperaturaService>();  
builder.Services.AddSingleton<RelatorioService>();  
// RelatorioService pede ITemperaturaService  
// no construtor
```

Testado: a app nem arranca

```
Cannot consume scoped service  
'ITemperaturaService' from singleton  
'RelatorioService'.
```

Se fosse permitido, o Singleton ficava com o Scoped do primeiro utilizador para sempre.

Registrar os serviços nos dois arranques

A RCL não tem arranque nem contentor. Quem regista são as apps, cada uma no seu, sempre antes do Build().

As mesmas linhas em MauiProgram.cs e em Program.cs

```
using UtilidadesRCL.Data.Interfaces;  
using UtilidadesRCL.Data.Services;  
  
builder.Services.AddScoped<IEnergiaService, EnergiaService>();  
builder.Services.AddScoped<IEventoService, EventoService>();  
builder.Services.AddScoped<INoticiaService, NoticiaService>();  
builder.Services.AddScoped<ITemperaturaService, TemperaturaService>();
```

Porquê nos dois

Cada app tem o seu contentor. Registrar só num lado faz a outra falhar ao abrir a página.

MauiProgram.cs

```
var builder =  
    MauiApp.CreateBuilder();  
// ...  
builder.Services  
    .AddMauiBlazorWebView();  
// registros aqui  
return builder.Build();
```

Program.cs

```
var builder = WebApplication  
    .CreateBuilder(args);  
builder.Services  
    .AddRazorComponents()  
    .AddInteractiveServerComponents();  
// registros aqui  
var app = builder.Build();
```

O erro quando falta

Cannot provide a value for property 'temperaturaService' ... There is no registered service of type 'ITemperaturaService'.

O que ganhamos: trocar a implementação

Mais à frente: os dados vêm da API (ficha 6)

```
public class TemperaturaApiService : ITemperaturaService
{
    private readonly HttpClient _http;

    public TemperaturaApiService(HttpClient http)
        => _http = http;

    public async Task<IEnumerable<Temperatura>>
        LoadTemperaturasAsync()
        => await _http.GetFromJsonAsync<
            IEnumerable<Temperatura>>("api/temperaturas") ?? [];
}
```

A única alteração no registo

```
// antes
AddScoped<ITemperaturaService,
    TemperaturaService>();

// depois
AddScoped<ITemperaturaService,
    TemperaturaApiService>();
```

- **O TemperaturaComponent não muda uma linha.**
- O contentor vê que TemperaturaApiService pede um HttpClient e cria-o primeiro, desde que esteja registado.
- É isto que acontece na ficha 6, quando os dados passarem a vir da API.

Compilar antes dos componentes

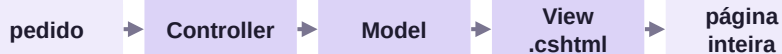
- 1 Pastas Data/DTO, Data/Interfaces, Data/Services e Pages
- 2 Quatro DTO: Energia, Evento, Noticia, Temperatura
- 3 Quatro interfaces e quatro serviços
- 4 Os quatro AddScoped no MauiProgram.cs
- 5 Os mesmos quatro no Program.cs
- 6 Os using da RCL no topo dos dois arranques

Teste

Compilar a solução (Ctrl+Shift+B): não deve haver erros. Ainda não há nada novo no ecrã: os serviços só aparecem quando houver componentes a usá-los, na etapa 4.

MVC ou Blazor: páginas ou componentes

MVC e Razor Pages (o que não usamos)



Cada clique ou formulário é um pedido novo e, normalmente, uma página nova.

- o servidor não guarda estado entre pedidos;
- o código (controller) e o HTML (view) estão em ficheiros separados;
- a página é gerada de uma vez e enviada inteira.

Blazor (o que usamos)

Um componente .razor

HTML
a marcação

C# (@code)
métodos e eventos

estado
ex.: temperaturas

algo muda:
um clique, dados
que chegam

o componente
redesenha-se e só
a parte que mudou
é atualizada

Onde corre o C#: na Web (Interactive Server), no servidor, com uma ligação que fica aberta; no MAUI, dentro da app, no dispositivo. Na ficha 6 voltam os controllers, mas só para devolver dados (JSON).

Anatomia de um componente com dados

```
@page "/temperaturas"
@Inject ITemperaturaService servico

<h3>Temperaturas</h3>
@if (temperaturas is null)
{
    <span>A carregar...</span>
}
else
{
    @foreach (var t in temperaturas)
    {
        <p>@t.Dia: @t.TemperaturaC °C</p>
    }
}

@code {
    private IEnumerable<Temperatura>? temperaturas;

    protected override async Task OnInitializedAsync()
    {
        temperaturas = await servico.LoadTemperaturasAsync();
    }
}
```

@page

Dá a rota. Sem ele, o componente não é página: usa-se dentro de outro.

@inject

Pede um serviço ao contentor. Fica disponível como propriedade.

@if · @foreach

C# a decidir o HTML. O `is null` cobre o tempo de espera.

@code

Campos e métodos da classe do componente.

OnInitializedAsync

Corre quando o componente é criado: o sítio para ir buscar dados.

Ciclo de vida: porque aparece o “A carregar”



Na app Web corre duas vezes. Com a pré-renderização, o servidor espera pelos dados e envia o HTML já com a tabela; quando o circuito liga, o componente é criado outra vez. Vê-se a tabela, o “A carregar” e a tabela de novo. É normal.

A etapa 4, por esta ordem

- 1 **TemperaturaComponent** /temperaturas, em conjunto, para fixar a forma 3e
- 2 **Energia, Evento, Notícia** o mesmo molde da Temperatura 3e
- 3 **MenuComponent e HomeComponent** peças sem @page; primeiro o menu 3e
- 4 **Home de cada app** passa a mostrar o HomeComponent 3e
- 5 **Entradas no menu** no NavMenu da RCL, que as duas apps usam 3e
- 6 **_Imports.razor da RCL** @using e @inject para todos os componentes 3f
- 7 **Router e Program.cs** as páginas da RCL passam a ter endereço antes da 3g

Sublinhados a vermelho?

Os componentes usam serviços e DTO através do `_Imports` da RCL (passo 6). Até lá, o Visual Studio sublinha-os.

É normal: compila-se na 3g. Quem preferir faz o passo 6 primeiro.

Com **@page** é uma página com endereço. Sem @page é uma peça que se usa como uma tag: `<MenuComponent />`

Pages/TemperaturaComponent.razor

o que se vê: rota e HTML

```
@page "/temperaturas"

<h3>Temperaturas para esta semana</h3>

@if (temperaturas is null)
{
    <span>A carregar temperaturas...</span>
}
else
{
    <table class="table table-striped">
        <thead>
            <tr><th>Dia</th><th>Temperatura (°C)</th><th>Resumo</th></tr>
        </thead>
        <tbody>
            @foreach (var t in temperaturas)
            {
                <tr>
                    <td>@t.Dia</td>
                    <td>@t.TemperaturaC</td>
                    <td>@t.Resumo</td>
                </tr>
            }
        </tbody>
    </table>
}
```

o que se faz: no fim do mesmo ficheiro

```
@code {
    private IEnumerable<Temperatura>? temperaturas;

    protected override async Task OnInitializedAsync()
    {
        temperaturas = await temperaturaService
            .LoadTemperaturasAsync();
    }
}
```

Repare-se

- Não há `@inject`: o `temperaturaService` vem do `_Imports.razor` da RCL.
- Não há `@rendermode`: o mesmo componente serve a Web e o MAUI.
- Não há `@using`: o DTO também vem do `_Imports`.
- `table table-striped` são classes do Bootstrap.

Duas formas de escrever o mesmo componente

Como na ficha: tudo no .razor

```
@page "/temperaturas"

<h3>...</h3>
@* ... a tabela ... *@

@code {
    private IEnumerable<Temperatura>? temperaturas;
    protected override async Task OnInitializedAsync()
    { ... }
}
```

É a mesma classe

partial diz ao compilador que a classe está repartida por dois ficheiros. É o mesmo que MainPage.xaml e MainPage.xaml.cs. O nome e o namespace têm de bater certo.

Alternativa: EnergiaComponent.razor

```
@page "/energias"

<h3>...</h3>
@* ... a tabela ... *@
```

...e EnergiaComponent.razor.cs (code-behind)

```
public partial class EnergiaComponent
{
    private IEnumerable<Energia>? energias;
    protected override async Task OnInitializedAsync()
    { ... }
}
```

O `_Imports` não chega ao `.cs`

Por isso o .razor.cs tem `using UtilidadesRCL.Data.DTO` no topo.

partial class: uma classe em dois ficheiros

Cada parte leva partial e o compilador junta-as numa só classe. Serve sobretudo para juntar código gerado por uma ferramenta com o código que escrevemos: quando a ferramenta volta a gerar a sua parte, a nossa não se perde.

Gerado a partir do .razor (não se vê)

```
public partial class EnergiaComponent
    : ComponentBase
{
    [Inject] IEnergiaService energiaService
    // + o HTML, convertido em código C#
}
```

+

Com code-behind (.razor.cs)

```
public partial class EnergiaComponent
{
    IEnumerable<Energia>? energias;

    protected override async Task
        OnInitializedAsync() { ... }
}
```

=

Uma só classe

- herda de **ComponentBase**
- tem o **energiaService** do **@inject**
- tem **energias** e **OnInitializedAsync**
- desenha o HTML do .razor

Também no MAUI: MainPage.xaml

O XAML gera uma parte com o **InitializeComponent()** e um campo para cada controlo com **x:Name**, como o **blazorWebView**. O **MainPage.xaml.cs** é a outra parte e usa esses campos como se fossem seus.

Se o namespace não coincidir

O .razor.cs tem de estar em **UtilidadesRCL.Pages**, o namespace que o .razor recebe da pasta. Senão são duas classes diferentes. Testado:

- CS0115 no **OnInitializedAsync**: não há nada para fazer override;
- sem o override, CS0103: o .cs não vê o **energiaService** e o .razor não vê o **energias**.

Pages/MenuComponent.razor: uma peça

UtilidadesRCL/Pages/MenuComponent.razor (completo)

```
<div class="d-flex flex-wrap gap-2 mt-3">  
  <NavLink class="btn btn-outline-primary" href="energias">Energia</NavLink>  
  <NavLink class="btn btn-outline-primary" href="eventos">Eventos</NavLink>  
  <NavLink class="btn btn-outline-primary" href="noticias">Notícias</NavLink>  
  <NavLink class="btn btn-outline-primary" href="temperaturas">Temperaturas</NavLink>  
</div>
```

Sem @page, é uma tag

Não tem endereço. Aparece onde se escrever `<MenuComponent />`, como se fosse HTML.

NavLink com aspeto de botão

`btn` e `btn-outline-primary` são classes do Bootstrap. O `NavLink` continua a navegar.

Precisa do Routing

O `NavLink` vive no namespace `...Components.Routing`, já no `_Imports` da RCL desde a etapa 2.

A seguir: o `HomeComponent` usa-o com `<MenuComponent />`. Por isso faz-se primeiro a peça.

Um Home partilhado, duas mensagens

UtilidadesRCL/Pages/HomeComponent.razor

```
<h1>Utilidades</h1>
```

```
<p>@MensagemBody</p>
```

```
<MenuComponent />
```

```
@code {  
    [Parameter]  
    public string? MensagemBody { get; set; }  
}
```

[Parameter]

Transforma a propriedade num atributo que quem usa o componente pode preencher. É assim que um componente recebe dados de fora.

MauiUtilidades/Components/Pages/Home.razor

```
@page "/"
```

```
@using UtilidadesRCL.Pages
```

```
<HomeComponent MensagemBody="Bem-vindo à versão MAUI das Utilidades." />
```

BlazorUtilidades/Components/Pages/Home.razor

```
@page "/"
```

```
@using UtilidadesRCL.Pages
```

```
<PageTitle>Utilidades</PageTitle>
```

```
<HomeComponent MensagemBody="Bem-vindo à versão Web das Utilidades." />
```

Porque é que o @page "/" fica nas apps

A página inicial é de cada app. A RCL dá o conteúdo; a app decide a rota e a mensagem.

O menu lateral com as utilidades

UtilidadesRCL/Shared/NavMenu.razor, bloco <nav> (o resto fica igual)

```
<nav class="flex-column">
  <div class="nav-item px-3">
    <NavLink class="nav-link" href="" Match="NavLinkMatch.All">
      <span class="bi bi-house-door-fill-nav-menu" aria-hidden="true"></span> Home
    </NavLink>
  </div>
  <div class="nav-item px-3">
    <NavLink class="nav-link" href="energias">
      <span class="bi bi-list-nested-nav-menu" aria-hidden="true"></span> Energia
    </NavLink>
  </div>
  <div class="nav-item px-3">
    <NavLink class="nav-link" href="eventos">
      <span class="bi bi-list-nested-nav-menu" aria-hidden="true"></span> Eventos
    </NavLink>
  </div>
  <div class="nav-item px-3">
    <NavLink class="nav-link" href="noticias">
      <span class="bi bi-list-nested-nav-menu" aria-hidden="true"></span> Notícias
    </NavLink>
  </div>
  <div class="nav-item px-3">
    <NavLink class="nav-link" href="temperaturas">
      <span class="bi bi-list-nested-nav-menu" aria-hidden="true"></span> Temperaturas
    </NavLink>
  </div>
</nav>
```

NavLink em vez de <a>

Gera um <a> e acrescenta a classe active quando o endereço coincide.

NavLinkMatch.All

Só no Home: sem ele, o href vazio coincide com todas as páginas.

Um só menu

As duas apps usam este NavMenu: as entradas escrevem-se uma vez.

_Imports.razor: a caixa de ferramentas

Cada ficheiro C# diz no topo que namespaces usa. Nos .razor, em vez de repetir os `@using` em todos, escrevem-se uma vez no `_Imports.razor` e valem para os componentes da mesma pasta e das subpastas.

Na vida real

Uma caixa de ferramentas na bancada: quem trabalha ali usa-a sem trazer a sua. Quem está noutra sala não lhe chega.

Quem vê o `_Imports.razor` da RCL

`Pages/TemperaturaComponent.razor`

vê

`Shared/NavMenu.razor`

vê

`Data/Services/TemperaturaService.cs`

não vê (.cs)

`MauiUtilidades` e `BlazorUtilidades`

não vê (outro projeto)

Quando se altera

`@using ...Data.DTO`

Pasta nova com classes, como os DTO desta ficha.

`@inject ITemperaturaService`

Serviço usado em todos os componentes (esta ficha).

`@using MudBlazor`

Biblioteca nova de componentes.

`@attribute [Authorize]`

Autenticação: todas as páginas da pasta pedem sessão iniciada.

Se faltar

CS0246: tipo desconhecido. RZ10012: o componente é tratado como uma tag HTML qualquer.

_Imports.razor: um diferente por projeto

Os `@using` e `@inject` de um `_Imports.razor` aplicam-se a todos os `.razor` da mesma pasta e subpastas.

UtilidadesRCL/_Imports.razor (completo)

```
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.AspNetCore.Components.Routing
@using UtilidadesRCL
@using UtilidadesRCL.Pages
@using UtilidadesRCL.Shared
@using UtilidadesRCL.Data.DTO
@using UtilidadesRCL.Data.Interfaces

@inject IEnergiaService energiaService
@inject IEventoService eventoService
@inject INoticiaService noticiaService
@inject ITemperaturaService temperaturaService
```

O `@inject` aqui poupa repetição

Os quatro serviços ficam em todos os componentes da RCL, mesmo nos que não precisam.

MauiUtilidades/Components/_Imports.razor (completo)

```
@using System.Net.Http
@using System.Net.Http.Json
@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.AspNetCore.Components.Web.Virtualization
@using Microsoft.JSInterop
@using MauiUtilidades
@using MauiUtilidades.Components
@using UtilidadesRCL.Shared
```

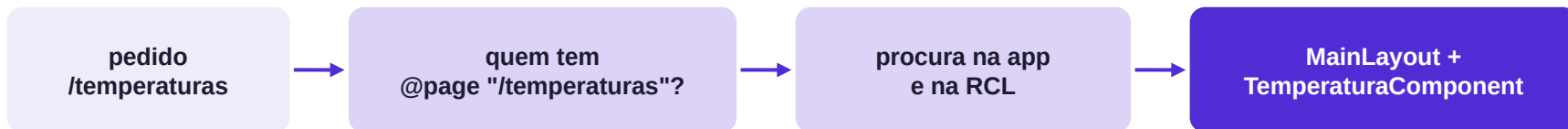
O do Web tem o mesmo `@using UtilidadesRCL.Shared` no fim.

O que o `_Imports` não faz

Não chega aos `.cs`, nem passa de um projeto para outro.

Routes.razor: o rececionista

O Routes.razor tem o Router: recebe o endereço pedido, procura o componente com esse @page e mostra-o dentro do layout por omissão, o MainLayout (menu e cabeçalho).



Ninguém tem: "não encontrado" (na Web, 404).

Na vida real

O rececionista de um edifício: pergunta-se por alguém, ele vê na lista em que gabinete está e leva lá a pessoa. Se o nome não está na lista, não há gabinete.

Mudam muitas vezes juntos: na autenticação, o `_Imports` ganha os `@using` de autorização e o Routes passa a verificar permissões. Um ensina as palavras, o outro aplica a regra.

Páginas noutro projeto (esta ficha)

O rececionista só conhece o seu edifício. O `AdditionalAssemblies` dá-lhe a lista do edifício ao lado: a RCL.

Login

O `RouteView` passa a `AuthorizeRouteView`: o rececionista pede o cartão antes de abrir as páginas com `[Authorize]`.

O router tem de encontrar as páginas da RCL

As páginas das utilidades estão agora na RCL, noutra DLL. O Router só procura classes com @page no assembly da app: é preciso indicar-lhe a RCL.

MauiUtilidades/Components/Routes.razor

```
<Router AppAssembly="@typeof(MauiProgram).Assembly"
        AdditionalAssemblies="new[] { typeof(UtilidadesRCL._Imports).Assembly }">
    <Found Context="routeData">
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(Layout.MainLayout)" />
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />
    </Found>
</Router>
```

BlazorUtilidades/Program.cs

```
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode()
    .AddAdditionalAssemblies(
        typeof(UtilidadesRCL._Imports)
        .Assembly);
```

BlazorUtilidades/Components/Routes.razor

```
<Router AppAssembly="typeof(Program).Assembly"
        AdditionalAssemblies="new[] { typeof(UtilidadesRCL._Imports).Assembly }">
    <Found Context="routeData">
        <RouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)" />
        <FocusOnNavigate RouteData="routeData" Selector="h1" />
    </Found>
</Router>
```

Testado: sem a linha do Program.cs, / dá 200 e /temperaturas dá 404.

Porquê duas vezes na Web

O Routes.razor trata da navegação com a app aberta. O Program.cs trata do primeiro pedido: escrever o endereço ou fazer F5.

A RCL completa, antes de compilar

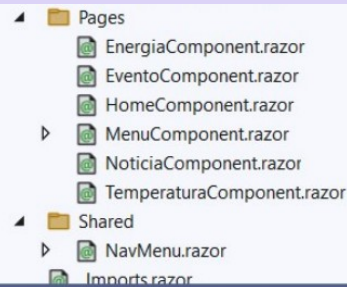


Figura 29 da ficha

Porquê comparar agora

Uma pasta ou um ficheiro em falta encontra-se mais depressa aqui do que pelos erros da compilação.

Confere com a figura

- **wwwroot:** o que veio do Web e os exemplos do template;
- **Data:** 4 DTO, 4 interfaces, 4 serviços;
- **Pages:** os seis componentes;
- **Shared:** o NavMenu que veio do MAUI;
- **_Imports.razor** com os `@using` e `@inject`.

Sem `Component1` nem `ExampleJsInterop.cs`.

Compilar e correr cada app

1

3g Compilar a solução inteira (Ctrl+Shift+B) e corrigir os erros.

2

3h Escolher BlazorUtilidades, perfil https, e correr: abre no browser.

3

3i Escolher MauiUtilidades e correr em Windows Machine.

4

3i Iniciar o emulador Android, esperar que arranque, recompilar e correr nele.

O que deve aparecer

- Home com a mensagem da app e os quatro botões do MenuComponent.
- Menu lateral com Home e as quatro utilidades.
- Cada página mostra “A carregar...” e, um segundo depois, a tabela.
- Bootstrap aplicado: tabelas às riscas e botões com cor.

Se algo falhar, o slide seguinte tem os sintomas e as causas.

Compila, mas não funciona?

Sintoma	Causa provável	Onde corrigir
Página sem Bootstrap	caminhos sem <code>_content/UtilidadesRCL/</code>	<code>index.html</code> e <code>App.razor</code>
CS0246: tipo ou namespace não encontrado	falta a referência de projeto ou um <code>@using</code>	<code>.csproj</code> da app ou <code>_Imports.razor</code>
RZ10012 para <code>NavMenu</code> ; menu vazio	o <code>NavMenu</code> mudou de namespace	<code>@using UtilidadesRCL.Shared</code> nas apps
RZ9985: Multiple components use the tag 'NavMenu'	ficou uma cópia do <code>NavMenu</code> numa app	apagá-la; fica só o da RCL
RZ10012 para <code>NavLink</code> ; links não navegam	falta o Routing na RCL	<code>_Imports.razor</code> da RCL
Cannot provide a value for property	serviço não registado nessa app	<code>MauiProgram.cs</code> ou <code>Program.cs</code>
Clicar no menu não abre a página	o Router não conhece a RCL	<code>AdditionalAssemblies</code> no <code>Routes.razor</code>
404 ao recarregar <code>/temperaturas</code> na Web	o servidor não conhece as rotas da RCL	<code>AddAdditionalAssemblies</code> no <code>Program.cs</code>
Aviso CS8602 no <code>await</code>	interface com <code>Task<...>?</code>	tirar o <code>?</code> da interface

Correr a Web e um device ao mesmo tempo

1

Escolher antes o device na barra: Windows Machine ou o emulador.

2

Clique direito na solução > Configurar Projetos de Inicialização.

3

Vários projetos de inicialização: Ação Iniciar em BlazorUtilidades e em MauiUtilidades.

4

F5: abrem o browser e a app ao mesmo tempo.

```
localhost:7xxx/temperaturas
```

Temperaturas

Segunda	24	Ameno
Terça	27	Quente

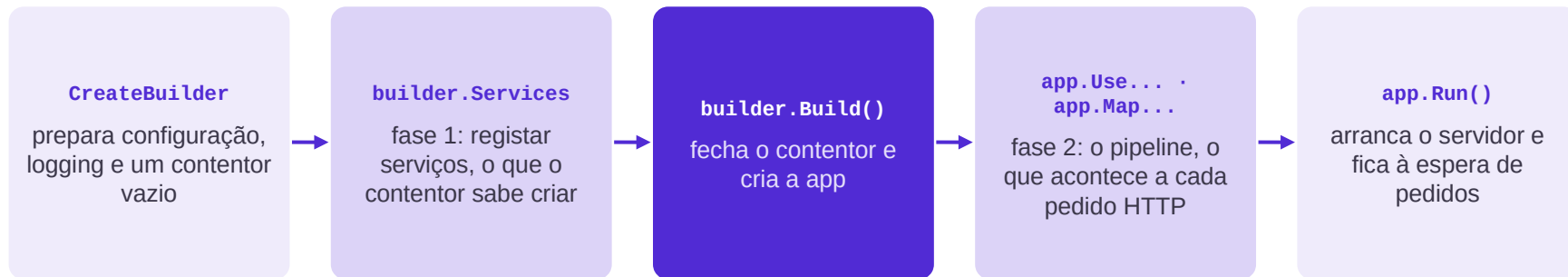
Temperaturas

Segunda	24
Terça	27

O mesmo TemperaturaComponent, dois anfitriões diferentes.

No arranque múltiplo, o device usado é o que estava selecionado antes de abrir a configuração.

Program.cs: o arranque em duas fases



Antes do Build(): o que existe

Regista-se tudo o que a app vai precisar: os componentes Razor, os serviços da RCL e, mais à frente, a base de dados e a autenticação. Depois do Build() já não se pode registar nada.

Depois do Build(): o que acontece

Monta-se o caminho de cada pedido: erros, HTTPS, ficheiros estáticos, proteção de formulários e, no fim, as páginas. A ordem conta.

BlazorUtilidades/Program.cs, completo

ficheiro final; a laranja, o que a parte 3 acrescentou

```
using BlazorUtilidades.Components;
using UtilidadesRCL.Data.Interfaces;
using UtilidadesRCL.Data.Services;
var builder = WebApplication.CreateBuilder(args);
// 1. Serviços: o que o contentor sabe criar
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();
builder.Services.AddScoped<IEnergiaService, EnergiaService>();
builder.Services.AddScoped<IEventoService, EventoService>();
builder.Services.AddScoped<INoticiaService, NoticiaService>();
builder.Services.AddScoped<ITemperaturaService, TemperaturaService>();
var app = builder.Build();
// 2. Pipeline: por onde passa cada pedido HTTP
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseAntiforgery();
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode()
    .AddAdditionalAssemblies(typeof(UtilidadesRCL._Imports).Assembly);
app.Run();
```

using ...

o Program.cs é C# normal: precisa dos namespaces

CreateBuilder(args)

configuração, logging e contentor

AddRazorComponents

regista o Blazor e o modo Server

AddScoped<...>

os quatro serviços da RCL

IsDevelopment()

em produção: erros e HSTS

MapRazorComponents

liga endereços a páginas, com a RCL

O pipeline: por onde passa cada pedido

1

UseExceptionHandler · UseHsts

só em produção: trata os erros (a página /Error foi apagada, dá 404); HSTS obriga a HTTPS

2

UseHttpsRedirection

um pedido http:// é redirecionado para https://

3

UseStaticFiles

se o pedido for um ficheiro do wwwroot ou de _content/, responde já e termina aqui

4

UseAntiforgery

protege os formulários contra pedidos forjados por outros sites

5

MapRazorComponents<App>

gera a página pedida a partir do App.razor e dos componentes com @page

A ordem conta

O UseStaticFiles vem antes das páginas: um .css ou uma imagem nunca chegam a passar pelo Blazor.

E a interatividade?

Depois de a página chegar, o blazor.web.js abre uma ligação SignalR ao servidor. Os cliques já não passam por este pipeline.

MauiUtilidades/MauiProgram.cs, completo

ficheiro final; a laranja, o que a parte 3 acrescentou

```
using Microsoft.Extensions.Logging;
using UtilidadesRCL.Data.Interfaces;
using UtilidadesRCL.Data.Services;
namespace MauiUtilidades
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureFonts(fonts =>
                {
                    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                });
            builder.Services.AddMauiBlazorWebView();
            builder.Services.AddScoped<IEnergiaService, EnergiaService>();
            builder.Services.AddScoped<IEventoService, EventoService>();
            builder.Services.AddScoped<INoticiaService, NoticiaService>();
            builder.Services.AddScoped<ITemperaturaService, TemperaturaService>();

            #if DEBUG
            builder.Services.AddBlazorWebViewDeveloperTools();
            builder.Logging.AddDebug();
            #endif

            return builder.Build();
        }
    }
}
```

CreateBuilder()

o mesmo padrão builder da Web

UseMauiApp<App>

a classe App (App.xaml) cria a janela

ConfigureFonts

fontes nativas de Resources/Fonts

AddMauiBlazorWebView

registra o que a BlazorWebView precisa

AddScoped<...>

os mesmos quatro serviços da RCL

#if DEBUG

só em debug: F12 na WebView e logs

Os dois arranques lado a lado

	Program.cs (Web)	MauiProgram.cs (MAUI)
Criar o builder	<code>WebApplication.CreateBuilder(args)</code>	<code>MauiApp.CreateBuilder()</code>
Registrar o Blazor	<code>AddRazorComponents()...</code>	<code>AddMauiBlazorWebView()</code>
Registrar serviços	<code>builder.Services.AddScoped<...>()</code>	<code>builder.Services.AddScoped<...>()</code>
Construir	<code>builder.Build()</code> devolve a app	<code>builder.Build()</code> devolve a MauiApp
Pipeline HTTP	<code>app.Use...</code> e <code>app.Map...</code>	não há: não há HTTP
Onde começa o Blazor	<code>App.razor</code> , com o <code><Routes></code>	<code>MainPage.xaml</code> > <code>index.html</code> > <code>Routes</code>
Quem arranca	<code>app.Run()</code>	a plataforma chama <code>CreateMauiApp()</code>

A linha igual nos dois é a dos serviços: é por ela que a RCL funciona dos dois lados.

Escrever uma vez, usar nos dois lados

RCL

O código comum vive numa biblioteca. As apps ficam com o arranque, o nativo e a página anfitriã.

wwwroot e _content/

Ficheiros estáticos, servidos tal como estão. Os da RCL chegam em _content/UtilidadesRCL/.

DTO e record

Só dados, para viajar entre partes. Um record escreve-os numa linha, com igualdade por valor.

async e await

Uma Task é a promessa de um resultado. O await espera por ela sem bloquear: a thread fica livre.

Acoplamento e DI

Depender de interfaces, não de classes. Os componentes pedem, cada app regista.

Tempos de vida

Singleton, Scoped, Transient. No servidor, Singleton é de todos. Nunca Singleton a depender de Scoped.

Arranque

Program.cs: serviços, Build(), pipeline, Run(). MauiProgram.cs: o mesmo, sem HTTP.